

HANDS_ON_LabLF_Logging_file

Lab LF: Logging & Debugging (file-only, ohne DB)

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login admin N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0 Fertiges Referenz-Lab auf der Canvas: “**Lab LF - Logging & Debugging (file)**” Live gebaut und verifiziert. **Kein DBCPConnectionPool / SQLite nötig.**

Unterschied zur DB-Variante (HANDS_ON_Logging.md): Im Fehlerpfad wird der Fehler nicht mehr durch `PutDatabaseRecord` (Tabelle “gibt_es_nicht”) provoziert, sondern **file-basiert**: ein `PutFile` schreibt in ein **nicht existierendes Verzeichnis** mit `Create Missing Directories = false` und scheitert dadurch garantiert. Lehrpunkt identisch: Fehler **sichtbar loggen** statt still verwerfen.

Zwei Ketten:

Kette 1 (Erfolg, INFO):

GetFile OK → UpdateAttribute → LogAttribute (INFO) → PutFile OK
(pipeline, geprueft_am) (Attribute ins nifi-app.log)

Kette 2 (Fehler, ERROR):

GetFile Fehler → PutFile (in nicht-existentes Verzeichnis) —failure—> LogAttribute (ERROR, Log Payload=true)

Pfade

Input OK: data/input/labLF
Input Fehler: data/input/labLF_err
Output OK: data/output/labLF_ok
Fehler-Ziel: data/output/labLF_FEHLT/x <- existiert BEWUSST NICHT (loest den Fehler aus)

Prozessoren

Kette 1 (Erfolgspfad)

GetFile OK - Input Directory = `$N/data/input/labLF` - File Filter = `.*\.csv`, Keep Source File = `false` - **Polling Interval** = `2 sec` (Default 30s : sonst dauert das Abholen bis zu 30s!)

UpdateAttribute : setzt eigene Attribute zum Sichtbarmachen - `pipeline = logging-demo` - `geprueft_am = ${now()}`

LogAttribute (INFO) : der “printf-Debug” für NiFi - Log Level = `info` - Log FlowFile Properties = `true` - (optional: Attributes to Log = `pipeline|geprueft_am`, Output Format = `Line per Attribute`)

PutFile OK - Directory = `$N/data/output/labLF_ok` - Create Missing Directories = `true`, Conflict Resolution = `replace` - `success/failure` terminieren

Kette 2 (Fehlerpfad)

GetFile Fehler - Input Directory = `$N/data/input/labLF_err`, File Filter = `.*\.csv` - Polling Interval = `2 sec`

PutFile (provoziert Fehler) : der file-basierte Fehlerauslöser - Directory = `$N/data/output/labLF_FEHLT/x` (Verzeichnis existiert nicht) - **Create Missing Directories** = `false` ← darum scheitert es - `success` terminieren, `failure` → **LogAttribute (ERROR)**

LogAttribute (ERROR) - Log Level = `error` - Log FlowFile Properties = `true` - **Log Payload** = `true` (loggt auch den Inhalt; nur für kleine Dateien/Debug) - `success` terminieren

Verdrahtung

Von	Relationship	Nach
GetFile OK	success	UpdateAttribute
UpdateAttribute	success	LogAttribute INFO
LogAttribute INFO	success	PutFile OK
PutFile OK	success, failure	terminieren
GetFile Fehler	success	PutFile (provoziert Fehler)
PutFile (provoziert Fehler)	success	terminieren
PutFile (provoziert Fehler)	failure	LogAttribute ERROR
LogAttribute ERROR	success	terminieren

Übung A : LogAttribute auf dem Erfolgspfad

```
cp $N/data/samples/orders.csv $N/data/input/labLF/
sleep 5
ls -l $N/data/output/labLF_ok/           # Datei da
grep -A 20 "LogAttribute" $N/logs/nifi-app.log | grep -iE "pipeline|geprueft_am" | tail
```

Im `nifi-app.log` erscheint ein LogAttribute-Block mit allen Attributen, u.a. `pipeline = logging-demo` und `geprueft_am`. So macht man den unsichtbaren FlowFile-Zustand sichtbar.

Übung B : Fehler sichtbar loggen statt still verwerfen

```
cp $N/data/samples/orders.csv $N/data/input/labLF_err/  
sleep 5  
grep -i error $N/logs/nifi-app.log | tail -5
```

- In der UI: **rotes Bulletin** an “PutFile (provoziert Fehler)”.
- Im Log: ein **ERROR** von PutFile (... output directory ... does not exist and Processor is configured not to create missing directories) **plus** der LogAttribute-ERROR-Block (mit Inhalt, weil Log Payload = true).

Lehrpunkt: statt `failure` blind zu auto-terminieren (Datei verschwindet lautlos) hängt man `LogAttribute/LogMessage` oder ein `PutFile` an : jeder Fehler bleibt nachvollziehbar.

Übung C : Data Provenance lesen

Rechtsklick auf **LogAttribute INFO** → **View data provenance** → jüngstes Event (ⓘ): - Tab **Attributes**: `pipeline`, `geprueft_am` sichtbar. - Tab **Content** → “View”: der CSV-Inhalt. - “Show Lineage”: Graph `GetFile` → `UpdateAttribute` → `LogAttribute` → `PutFile`.

Verifiziertes Ergebnis (im Live-Test bestätigt)

- Erfolgspfad: `orders.csv` in `data/output/labLF_ok/`, INFO-LogAttribute-Block mit `pipeline/geprueft_am`.
- Fehlerpfad: PutFile routet zu `failure` (`output directory ... does not exist`), LogAttribute loggt auf **ERROR**-Level.

Reset

```
find $N/data/input/labLF $N/data/input/labLF_err $N/data/output/labLF_ok -type f -delete  
# data/output/labLF_FEHLT NICHT anlegen (sonst scheitert Kette 2 nicht mehr)  
# PG stoppen
```

Häufige Stolpersteine

- **Datei wird ~30s nicht abgeholt** → `GetFile-Property Polling Interval` steht auf 30s; auf `2 sec` setzen.
- **Kette 2 scheitert nicht (keine ERROR-Logs)** → das Fehler-Zielverzeichnis existiert doch, oder `Create Missing Directories = true`. Verzeichnis löschen / Property auf `false`.
- **Nichts im Log trotz LogAttribute** → Log Level zu niedrig gewählt, oder im `nifi-app.log` nach dem falschen Begriff gesucht; nach `LogAttribute` greppen.
- **Log Payload zu groß** → `Log Payload = true` nur für kleine Demo-Dateien, sonst flutet es das Log.